

Hybrid Lexical–Semantic Retrieval for Tool Selection in Agent Systems — Conscious Engines

 consciousengines.com/blog/hybrid-lexicalsemantic-retrieval-for-tool-selection-in-agent-systems

Conscious Engines

April 30, 2026



When an agent has five tools, tool selection is a prompt engineering problem. When it has thousands, it becomes search infrastructure.

Picture an agent wired into a platform like Composio: thousands of executable actions across SaaS, CRM, finance, analytics, communication, and developer tools, all sitting in one catalog. A user types: "pull failed payments from Stripe, file a support ticket in Linear, and notify the on-call channel in Slack." Three sentences, three providers, half a dozen candidate tools spread across the catalog. Before the agent can do anything, it has to find them.

That moment — natural-language intent meeting a large action space — is the routing problem. The same shape shows up in MCP server registries, internal API catalogs, enterprise automation systems, SDK wrappers, database procedures, and workflow engines. As soon as user requests have to map onto a large set of executable actions, the tool router becomes a retrieval system.

This post describes a retrieval design for that setting: a self-hosted first-stage router that combines lexical matching, offline semantic expansion, dense retrieval, and rank fusion. The goal is not to make an LLM stare at every tool. It is to cheaply produce a high-quality candidate set that an agent or planner can use.

Why Tool Search Is Weird

Tool retrieval is not ordinary document retrieval.

In web search, the target document is often long enough to contain many ways of describing itself. API tools are sparse. A tool may have a short name, a terse description, and a schema written for developers rather than end users.

A user might ask, "find the customer's CRM record." The correct tool might talk about contacts, accounts, leads, or objects. Another user might ask to "notify the team," while the tool catalog says message, post, send, create, or publish. The intent and the API surface are semantically close but lexically far apart.

The problem is hard for three reasons. Tool names are optimized for API developers rather than natural-language users. Different providers use different words for the same operation. And multi-step tasks often pull tools from several providers at once.

The last point matters. Look back at the opening request. A router that finds one plausible email tool can look decent on simple queries and still fail on the Stripe → Linear → Slack workflow, because no single tool covers it end to end. Tool routing has to optimize for both: a first relevant hit, and complete workflow coverage.

The System Shape

The retriever combines four signals: BM25 over tool names and provider names, BM25 over descriptions and schema-derived text, BM25 over offline semantic expansions, and dense retrieval over natural-language tool documents.

Each signal produces its own ranked list. The final ranking is built with weighted Reciprocal Rank Fusion. This keeps the system interpretable: exact lexical matches, schema evidence, generated user-language phrases, and vector semantics all contribute separately.

The architecture is catalog-agnostic. A tool catalog only needs a stable tool identifier plus whatever descriptive fields are available: name, provider, description, parameter schema, examples, categories, or usage text. The same retrieval strategy can be applied to SaaS tools, internal APIs, MCP tools, or enterprise automations.

Stage 1: Separate Lexical Signals

BM25 remains a strong starting point because API catalogs contain many exact names and domain terms. It rewards rare matching terms, accounts for repeated terms without letting frequency dominate completely, and normalizes for document length. Those are

useful properties for API catalogs, where a provider name or operation verb can be highly informative.

We split lexical retrieval into three documents per tool. The name document captures tool names, provider names, operation verbs, and category hints. The description document captures tool descriptions, parameter names, parameter descriptions, schema text, and examples. The expansion document captures generated user-language phrases and domain synonyms.

This separation matters because the fields behave differently. Names are short and precise. Descriptions are broad but noisy. Expansions are phrased like user requests. If these are concatenated into one document, long schema text can dilute exact name evidence, and generated phrases can overpower precise API terms.

For the lexical-only configuration, each BM25 index is normalized independently and combined with fixed weights: 0.35 for the name index, 0.35 for the description index, and 0.30 for the expansion index. This gives exact names and schema evidence roughly equal weight, while still giving generated user-language phrases a substantial role.

Stage 2: Offline Semantic Expansion

The biggest vocabulary gap is not solved by tokenization. It is solved by adding the words users actually say.

For each tool, we generate expansion text before query time: when to use the tool, common user phrases, domain synonyms, related concepts, and task categories. A contact-search API might gain phrases such as "find a customer," "look up a client by email," or "search CRM contacts." A payment API might gain "failed invoice," "charge lookup," or "customer billing history."

This is a useful place to use an LLM because the work is amortized. The model is not called for every user request. It is used to enrich the catalog once, then the generated phrases become ordinary retrieval text.

That design has two advantages: query-time latency stays close to ordinary search, and the expansion remains inspectable, debuggable, and reusable across retrieval strategies.

It also avoids a common failure mode: asking an LLM to choose from thousands of raw tools at query time. The LLM is better used to bridge vocabulary offline than to brute-force a large catalog online.

Stage 3: Dense Retrieval

BM25 is strong when words overlap. Dense retrieval helps when the match is semantic.

The useful matches are often pairs like "customer" and "contact," "notify" and "send message," "expenses" and "accounting," or "issue tracker" and "project management."

For the reported hybrid run, we embed query text and tool text with BAAI bge-base-en-v1.5. The dense tool document is written as natural prose: provider, tool name, description, category hints, generated usage phrases, and domain synonyms. Embeddings are L2-normalized, so ranking can use dot-product similarity.

Dense retrieval is not a replacement for lexical search. It is a complementary signal. Exact tool names, provider names, and operation verbs still matter.

Stage 4: Rank Fusion

The hybrid retriever does not merge raw BM25 scores and dense similarities directly. Their scales are not comparable. BM25 scores depend on term frequency, document length, and corpus statistics. Dense scores are vector similarities.

Instead, each signal returns a ranked list. We then combine ranks using weighted Reciprocal Rank Fusion:

$$\sum_{j=1}^m \frac{w_j}{k + r_j(d)}$$

Here m is the number of retrievers, w_j is the weight for retriever j , $r_j(d)$ is the rank of document d under retriever j , and k controls how quickly lower ranks decay.

The reported hybrid run fuses four ranked lists: Name BM25, Description BM25, Expansion BM25, and Dense retrieval.

The best weighting from the sweeps was:

Signal	Weight
Name BM25	1
Description BM25	1
Expansion BM25	2
Dense retrieval	2

This weighting reflects the shape of the hard cases. Exact lexical evidence is still useful, but the hardest misses usually come from vocabulary mismatch. Upweighting expansion and dense retrieval improves semantic coverage without discarding exact matches.

Evaluation Metrics

We evaluate top- K retrieval with $K = 10$. For query i , R_i^K is the ordered list of returned tools, P_i is the set of required primary tools, and A_i is the set of all acceptable tools including valid alternatives. For no-tool cases, P_i and A_i are empty; this benchmark evaluates candidate retrieval rather than abstention.

Metric	Definition	Why it matters
Recall@10	Whether at least one acceptable tool appears in the top 10.	Measures whether the planner gets any useful candidate.
MRR@10	Reciprocal rank of the first acceptable tool, averaged across queries.	Rewards putting the right tool early, not just somewhere in the list.
Full Recall@10	Fraction of required primary tools covered in the top 10.	Captures whether multi-step tasks have enough tools to proceed.
Multi-tool Recall@10	Full Recall@10 restricted to queries where $ P_i > 1$.	Isolates workflow tasks where one good hit is not enough.
Avg Latency	Mean wall-clock time to return the ranked top-10 list after initialization.	Measures overhead on the agent's critical path.

Results

The benchmark contains named-provider queries, generic category queries, cross-provider workflow queries, ambiguous provider choices, and tasks where no tool is appropriate.

The headline result:

Configuration	Recall@10	MRR@10	Full Recall@10	Multi-tool Recall@10	Avg Latency
Lexical BM25	0.642	0.408	0.405	0.269	111ms
Hybrid rank fusion	0.736	0.472	0.477	0.344	120ms

The important movement is not just Recall@10. MRR improves, meaning correct tools move earlier in the list. Full Recall and Multi-tool Recall also improve, which matters for the

kind of workflow we opened with — where the router has to surface tools across several providers in a single shot, not just the first plausible match.

The latency increase is small because the semantic work is mostly shifted out of the query path. The online stage is retrieval and rank fusion, not a fresh LLM call over the catalog.

What Generalizes

The design is not tied to one vendor catalog.

It generalizes whenever tools have stable identifiers, names or operation labels, provider or namespace information, descriptions or schemas, and optional examples, categories, or usage hints.

That covers hosted tool catalogs, MCP registries, internal service catalogs, workflow automation actions, SDK wrappers, and enterprise API gateways.

The retrieval recipe is also modular. If a catalog has rich examples, the description index becomes stronger. If descriptions are sparse, dense retrieval and generated usage phrases matter more. If provider names are important, the name index can be weighted more heavily. That modularity is the point: the system is intentionally not a single monolithic embedding search.

Lessons

First, tool routing is mostly vocabulary translation. Users describe outcomes. APIs describe operations. Bridging that gap before query time is one of the highest-leverage improvements.

Second, field separation matters. A tool name, a schema description, and a generated usage phrase are not the same kind of evidence. Treating them as separate retrievers produces better ranking behavior than flattening everything.

Third, rank fusion is a practical way to combine heterogeneous signals. It avoids brittle score normalization while still rewarding tools that appear near the top of multiple independent rankings.

Finally, the retrieval layer should stay boring. The agent can be creative later. The router's job is to be fast, measurable, and hard to fool: take a natural-language task, search a large action space, and return the tools most likely to make the next step possible.

Keep reading

more in research

Kinetic-4B: A 4-Billion Parameter Model That Outperforms Claude Haiku at Tool Calling
